Advance Educational Activities Pvt. Ltd.

Unit 6: Graphs

6.1 — Introduction to Graphs & Terminology

Graphs are the most general and powerful data structure in Computer Science.
If trees model hierarchy, graphs model relationships.

This unit builds the *mental model* required to understand networks, maps, dependencies, flows, and connections.

1️⃣ What is a Graph?

Definition

A graph is a non-linear data structure consisting of:

- Vertices (nodes)
- Edges (connections between nodes)

Formally:

Graph G = (V, E)

V → set of vertices

E → set of edges

Unlike trees:

- Graphs can have cycles
- Graphs do not require a root
- Graphs can be disconnected
- Nodes can have multiple relationships

Key idea:

A graph is a way to model any system where things are connected.

2️⃣ Why Trees Are Not Enough

Trees are powerful, but **limited**.

Limitations of Trees

- Only one parent per node
- No cycles allowed
- Strict hierarchy
- Cannot represent many-to-many relationships

© Advanced Educational Activities Pvt. Ltd. 95B, Siddamsetty Complex, Park Lane, Secunderabad – 500003. All rights reserved. No part of this material may be reproduced, in any form or by any means, without permission in writing. This course material is only for the use of bonafide students of **Advanced Educational Activities Pvt. Ltd. (T.I.M.E.)** and its licensees/franchisees and is not for sale. (47 pages) (cael/caem) SM2132506/1

Graphs Solve This

- Nodes can have multiple connections
- Cycles are allowed
- Relationships can be **bidirectional**
- Multiple paths between nodes possible

Example:

- Family tree → Tree
- Social network → Graph

3️⃣ Graph vs Tree (Conceptual Comparison)

Aspect	Tree	Graph
Structure	Hierarchical	Network-based
Root	Mandatory	Not required
Cycles	Not allowed	Allowed
Connectivity	Always connected	May be disconnected
Parent rule	One parent	Multiple connections
Use case	Hierarchy	Relationships

Every tree is a graph, but not every graph is a tree.

4️⃣ Graph Components

Vertex (Node)

- Represents an **entity**
- Examples: user, city, router, webpage

Edge

- Represents a relationship or connection
- Can be:
  - Directed
  - Undirected
  - Weighted

Example:

A —— B

A and B are vertices, the line is an edge.

5️⃣ Basic Graph Terminology

Let's lock these terms clearly — these words repeat everywhere.

Degree

- Number of edges connected to a vertex
- In undirected graph:
  Degree(v) = number of connected edges

Path

- A sequence of vertices connected by edges

Example:

A → B → C → D

Cycle

- A path that starts and ends at the **same vertex**

Example:

A → B → C → A

Connected Graph

- Every vertex is reachable from every other vertex

Disconnected Graph

- At least one vertex is **not reachable** from others

Self-loop

- An edge from a vertex to itself

Example:

A → A

Parallel Edges

- More than one edge between the same pair of vertices

6 Directed vs Undirected Graphs

Undirected Graph

- Edges have **no direction**
- Relationship is mutual

Example:

A —— B

Use cases:

- Friendships
- Road networks (two-way)

Directed Graph (Digraph)

- Edges have **direction**

Example:

A → B

Use cases:

- Twitter follow
- One-way streets
- Task dependencies

7️⃣ Weighted vs Unweighted Graphs

Unweighted Graph

- All edges are equal
- Only connectivity matters

Example:

A —— B

Weighted Graph

- Each edge has a **cost / weight**

Example:

A ——(5)—— B

Use cases:

- Distance between cities
- Network latency
- Cost optimization problems

8️⃣ Simple Graph vs Multigraph

Simple Graph

- No self-loops
- No parallel edges

Multigraph

- Self-loops allowed
- Parallel edges allowed

Multigraphs model **real systems** more accurately (like transport networks).

9️⃣ Graph Classification (Big Picture)

Graphs can be:

- Directed / Undirected
- Weighted / Unweighted
- Cyclic / Acyclic
- Connected / Disconnected
- Simple / Multigraph

Trees are just a special case of graphs:

- Connected
- Acyclic
- N nodes → N-1 edges

🔟 Why Graphs Matter (Real-World Motivation)

Graphs are everywhere:

Social Networks

- Users → vertices
- Friendships / follows → edges

Road Maps

- Cities → vertices
- Roads → edges
- Distance → weights

Internet & Networking

- Routers → vertices
- Connections → edges
- Latency → weights

Dependency Systems

- Tasks → vertices
- Dependencies → directed edges

Recommendation Engines

- Users ↔ Products
- Relationships drive suggestions

1️⃣1️⃣ Mental Model (Very Important)

Think of a graph as:

- A map of relationships

- A network of possibilities
- A system of choices and paths

Every graph problem is about:

- Reaching
- Connecting
- Optimizing
- Traversing
- Detecting cycles
- Finding shortest paths

Quick Summary

- Graph = vertices + edges
- Used to model **relationships**
- More powerful than trees
- Supports cycles, multiple paths, and disconnections
- Comes in many forms: directed, weighted, simple, multigraph
- Foundation for:
  - BFS, DFS
  - Shortest paths
  - Network algorithms
  - Real-world systems

6.2 — Types of Graphs (Conceptual)

A graph is not one thing — it is a family of structures.

1️⃣ Why So Many Types of Graphs?

A graph's **type** depends on:

- Direction of edges
- Presence of weights
- Connectivity
- Cycles
- Special constraints

Each type models different real-world problems.

Choosing the correct graph type = 50% of problem solving.

2️⃣ Undirected Graph

Definition

Edges have **no direction**.
If A is connected to B, then B is also connected to A.

```
A ── B
|    |
C ── D
```

Properties

- Edge (A, B) = (B, A)
- Degree = number of connected edges

Real-World Examples

- Facebook friendships
- Undirected road maps
- Peer-to-peer networks

3️⃣ Directed Graph (Digraph)

Definition

Edges have a **direction**.

```
A → B → C
↑       ↓
└───────┘
```

Properties

- (A → B) ≠ (B → A)
- Two types of degree:
  - In-degree
  - Out-degree

Real-World Examples

- Twitter follow system
- Web page links
- Task dependency graphs

4️⃣ Weighted Graph

Definition

Each edge has a weight (cost/value).

```
A --5-- B
|       |
2       3
|       |
C --4-- D
```

Properties

- Weight may represent:
  - Distance
  - Cost
  - Time
  - Bandwidth

Real-World Examples

- Google Maps (distance/time)
- Network routing
- Airline ticket costs

5️⃣ Unweighted Graph

Definition

All edges are treated equally (weight = 1).

A —— B —— C

Properties

- Focus is on **connectivity**, not cost
- Often used in traversal problems

Real-World Examples

- Social connections
- Friendship graphs
- Basic network reachability

6️⃣ Cyclic Graph

Definition

A graph that contains **at least one cycle**.

```
A  → B →  C
↑         ↓
└─────────┘
```

Properties

- Path starts and ends at same vertex
- Cycles may be directed or undirected

Real-World Examples

- Traffic routes
- Circular dependencies
- Feedback systems

7️⃣ Acyclic Graph

Definition

A graph with **no cycles**.

A → B → C → D

Special Case

- DAG (Directed Acyclic Graph) — very important

Real-World Examples

- Task scheduling
- Compiler dependency graphs
- Version control history

8️⃣ Connected Graph

Definition

Every vertex is reachable from every other vertex.

```
A ── B ── C
|          |
D ———————— E
```

Properties

- Single connected component
- No isolated nodes

Real-World Examples

- Fully reachable network
- Power grids

9️⃣ Disconnected Graph

Definition

Graph with multiple disconnected components.

A —— B C —— D

Properties

- At least one node is unreachable from another
- More than one connected component

Real-World Examples

- Broken networks
- Islanded systems

🔟 Complete Graph

Definition

Every vertex is connected to **every other vertex**.

For n nodes:

Edges = n(n−1)/2

Properties

- Maximum number of edges
- Very dense graph

Real-World Examples

- Fully connected mesh networks
- Small team communication models

1️⃣1️⃣ Bipartite Graph (Intro Only)

Definition

Vertices can be divided into **two disjoint sets**
Edges only connect vertices from **different sets**.

```
U-set: A   B
        \ /
         X
        / \
V-set: C   D
```

Properties

- No odd-length cycles
- Very important in matching problems

Real-World Examples

- Student ↔ Course assignment
- Job seekers ↔ Jobs
- Buyers ↔ Sellers

1 2 Tree as a Special Graph

A **Tree** is actually a **special graph**.

Tree Properties Recap:

- Connected
- Acyclic
- n nodes → n−1 edges

Tree ⊂ Graph

Key Insight

Every tree is a graph
BUT not every graph is a tree

1 3 Simple Graph vs Multigraph

Simple Graph

- No self-loops
- No parallel edges

Multigraph

- Parallel edges allowed
- Self-loops allowed

A == B (parallel edges)

A → A (self loop)

Usage

- Transport systems
- Multiple communication channels

1 4 Visual Summary Map

```
Graphs
 ├── Directed / Undirected
 ├── Weighted / Unweighted
 ├── Cyclic / Acyclic
 ├── Connected / Disconnected
 ├── Complete
 ├── Bipartite
 ├── Tree (Special Case)
 └── Simple / Multigraph
```

Quick Summary — 6.2

- Graphs come in many structural forms
- Type depends on:
  - Direction
  - Weight
  - Connectivity
  - Cycles
- Real-world problems demand **specific graph types**
- Trees are a **subset** of graphs
- Understanding graph types is essential **before algorithms**

6.3 — Graph Representations

A graph is only as powerful as how you store it.

1️⃣ Why Graph Representation Matters

Graphs can be **sparse** or **dense**, **small** or **huge**.
The way we represent a graph directly affects:

- Memory usage
- Speed of operations
- Ease of traversal
- Algorithm performance (BFS, DFS, Dijkstra, etc.)

Key idea:

Same graph → different representations → different efficiency.

2️⃣ Two Standard Graph Representations

There are two fundamental ways to store a graph:

1. Adjacency Matrix
2. Adjacency List

Every graph algorithm is built on top of **one of these two**.

Adjacency Matrix Representation

3️⃣ What Is an Adjacency Matrix?

An **Adjacency Matrix** is a **2D array** of size $V \times V$,

where V = number of vertices.

Rule:

matrix[i][j] = 1 → edge exists from i to j

matrix[i][j] = 0 → no edge

4️⃣ Structure (Undirected Graph)

Graph:

```
0 ── 1
|    |
2 ── 3
```

Adjacency Matrix:

```
    0 1 2 3
0 [ 0 1 1 0 ]
1 [ 1 0 0 1 ]
2 [ 1 0 0 1 ]
3 [ 0 1 1 0 ]
```

Undirected graph → matrix is **symmetric**.

5️⃣ Directed Graph (Adjacency Matrix)

For a directed edge $u \rightarrow v$:

matrix[u][v] = 1

Example:

0 → 1 → 2

Matrix:

```
    0 1 2
0 [ 0 1 0 ]
1 [ 0 0 1 ]
2 [ 0 0 0 ]
```

Not symmetric.

6️⃣ Weighted Graph (Adjacency Matrix)

Instead of 1, store **weight**:

matrix[i][j] = weight

matrix[i][j] = 0 or ∞ → no edge

Example:

0 --5-- 1

Matrix:

```
    0 1
0 [ 0 5 ]
1 [ 5 0 ]
```

7 Java Code — Adjacency Matrix

```
class GraphMatrix {
    int V;
    int[][] adj;

    GraphMatrix(int V) {
        this.V = V;
        adj = new int[V][V];
    }

    void addEdge(int u, int v) {
        adj[u][v] = 1;
        adj[v][u] = 1; // remove for directed graph
    }

    void printGraph() {
        for (int i = 0; i < V; i++) {
            for (int j = 0; j < V; j++) {
                System.out.print(adj[i][j] + " ");
            }
            System.out.println();
        }
    }
}
```

8 Complexity — Adjacency Matrix

Aspect	Complexity
Space	$O(V^2)$
Check edge (u,v)	O(1)
Add edge	O(1)
Iterate neighbors	O(V)

9 Advantages & Limitations

Advantages

- Simple to implement
- Constant-time edge lookup

- Best for **dense graphs**

Limitations

- Huge memory wastage for sparse graphs
- Iterating neighbors is slow

Adjacency List Representation

🔟 What Is an Adjacency List?

An Adjacency List stores, for each vertex,
a list of all its adjacent vertices.

Structure:

Vertex → List of neighbors

1️⃣1️⃣ Structure (Undirected Graph)

Graph:

```
0 ── 1
|    |
2 ── 3
```

Adjacency List:

```
0 → 1, 2
1 → 0, 3
2 → 0, 3
3 → 1, 2
```

Only stores **actual edges** → memory efficient.

1️⃣2️⃣ Directed Graph (Adjacency List)

Edge `u → v`:

u → v

Example:

0 → 1 → 2

List:

```
0 → 1
1 → 2
2 → (empty)
```

1 3 Weighted Graph (Adjacency List)

Store pairs (neighbor, weight).

0 → (1,5), (2,3)

1 4 Java Code — Adjacency List

```
import java.util.*;

class GraphList {
    int V;
    ArrayList<ArrayList<Integer>> adj;

    GraphList(int V) {
        this.V = V;
        adj = new ArrayList<>();

        for (int i = 0; i < V; i++)
            adj.add(new ArrayList<>());
    }

    void addEdge(int u, int v) {
        adj.get(u).add(v);
        adj.get(v).add(u); // remove for directed graph
    }

    void printGraph() {
        for (int i = 0; i < V; i++) {
            System.out.print(i + " → ");
            for (int v : adj.get(i))
                System.out.print(v + " ");
            System.out.println();
        }
    }
}
```

1 5 Complexity — Adjacency List

Aspect	Complexity
Space	O(V + E)
Add edge	O(1)
Check edge	O(deg(v))
Iterate neighbors	O(deg(v))

1 6 Advantages & Limitations

Advantages

- Memory efficient
- Ideal for **sparse graphs**
- Faster traversal

Limitations

- Edge lookup slower than matrix
- Slightly more complex implementation

Adjacency Matrix vs Adjacency List

1 7 Comparison Table

Feature	Adjacency Matrix	Adjacency List
Space	$O(V^2)$	$O(V + E)$
Edge lookup	$O(1)$	$O(deg(v))$
Memory usage	High	Low
Best for	Dense graphs	Sparse graphs
Traversals	Slower	Faster
Implementation	Easy	Moderate

1 8 When to Use Which?

Use Adjacency Matrix when:

- **Graph is dense**
- Fast edge checking is required
- Small number of vertices

Use Adjacency List when:

- Graph is sparse
- Traversals (BFS/DFS) are frequent
- Memory efficiency matters

Industry standard:

Most real-world systems use **Adjacency Lists**.

1 9 Conceptual Memory View

- Adjacency Matrix → Grid-based memory
- Adjacency List → Pointer-based memory

Graph performance = Representation + Algorithm.

Quick Summary — 6.3

- Graphs can be represented in **two ways**
- Matrix → fast lookup, high memory
- List → memory efficient, traversal-friendly
- Choice of representation affects algorithm performance
- BFS & DFS will operate on **both representations**

6.4 — Graph Traversal: Breadth-First Search (BFS)

Level-by-level exploration of a graph

1 What Is Graph Traversal?

Graph traversal means visiting every vertex of a graph in a systematic way.

Unlike trees:

- Graphs can have **cycles**
- Graphs may be **disconnected**
- A node can be reached via **multiple paths**

So traversal must be **carefully controlled**.

2 Why Do We Need Traversal in Graphs?

Traversal helps us to:

- Visit all vertices
- Explore connectivity
- Find shortest paths (unweighted graphs)
- Detect cycles
- Solve real-world problems like:
  - Social network reach
  - Routing
  - Broadcasting messages

Traversal = foundation of all graph algorithms

3️⃣ What Is BFS? (Core Idea)

Breadth-First Search (BFS) explores the graph:

Level by level, starting from a source vertex

It visits:

- First → all immediate neighbors
- Then → neighbors of neighbors
- Then → next level, and so on

This is why BFS is also called **Level Order Traversal of Graphs**

4️⃣ BFS Real-World Intuition

Example: Social Network

- You → level 0
- Your friends → level 1
- Friends of friends → level 2

BFS explores people **closest to you first**.

Example: Shortest Path (Unweighted)

- BFS guarantees the minimum number of edges to reach a node

5️⃣ Data Structures Used in BFS

BFS uses two essential tools:

1. **Queue** → to maintain level order
2. **Visited array / set** → to avoid revisiting nodes (cycles!)

Queue enforces FIFO → perfect for level-wise traversal

6️⃣ BFS Algorithm (High-Level Steps)

Given a starting vertex `src`:

1. Create a **visited[]** array (initially false)
2. Create an empty **queue**
3. Mark src as visited
4. Enqueue src
5. While queue is not empty:
    - Dequeue a vertex u
    - Process u

- For each neighbor v of u:
  - If not visited:
    - Mark visited
    - Enqueue v

7️⃣ BFS Using Adjacency List (Java Implementation)

Graph Representation (Adjacency List)

```
import java.util.*;

class Graph {
    int V;
    ArrayList<ArrayList<Integer>> adj;

    Graph(int V) {
        this.V = V;
        adj = new ArrayList<>();
        for (int i = 0; i < V; i++)
            adj.add(new ArrayList<>());
    }

    void addEdge(int u, int v) {
        adj.get(u).add(v);
        adj.get(v).add(u); // undirected graph
    }
```

BFS Code

```
    void bfs(int src) {
        boolean[] visited = new boolean[V];
        Queue<Integer> q = new LinkedList<>();
        visited[src] = true;
        q.add(src);

        while (!q.isEmpty()) {
            int node = q.poll();
            System.out.print(node + " ");
            for (int neighbor : adj.get(node)) {
                if (!visited[neighbor]) {
                    visited[neighbor] = true;
                    q.add(neighbor);
                }
            }
        }
    }
}
```

8️⃣ Dry Run with Example

Graph:

```
0 ── 1 ── 3
|     |
2     4
```

Adjacency List:

```
0 → 1, 2
1 → 0, 3, 4
2 → 0
3 → 1
4 → 1
```

BFS starting from 0

Step	Queue	Visited	Output
Start	[0]	0	
1	[1,2]	0,1,2	0
2	[2,3,4]	0,1,2,3,4	1
3	[3,4]	all	2
4	[4]	all	3
5	[]	all	4

BFS Output:

0 1 2 3 4

9️⃣ BFS Traversal Diagram (Level-wise)

```
Level 0:        0
               |
Level 1:     1   2
              |
Level 2:     3   4
```

BFS completes entire level before moving deeper

🔟 Handling Cycles (Why visited[] is Critical)

Without visited[]:

- Graphs with cycles → **infinite loop**
- Same node visited multiple times

Visited array ensures:

- Each node is processed **once**
- BFS runs efficiently

1️⃣1️⃣ BFS in Disconnected Graphs

If graph is **disconnected**, BFS from one source won't cover all nodes.

Solution:

```
for (int i = 0; i < V; i++) {
    if (!visited[i]) {
        bfs(i);
    }
}
```

This ensures full graph traversal

1️⃣2️⃣ Time & Space Complexity

Let:

- V = number of vertices
- E = number of edges

Time Complexity

O(V + E)

Each vertex and edge is visited once.

Space Complexity

O(V)

For:

- visited array
- queue

1️⃣3️⃣ When to Use BFS?

Use BFS when:

- Shortest path in **unweighted graph**
- Level-wise processing
- Minimum number of steps
- Broadcasting / spreading problems
- Graph is wide rather than deep

1 4 BFS vs DFS (Quick Preview)

Feature	BFS	DFS
Data structure	Queue	Stack / Recursion
Traversal	Level-wise	Depth-wise
Shortest path	Yes (unweighted)	No
Memory	Higher	Lower
Use case	Distance, levels	Cycles, paths

1 5 Real-World Applications of BFS

- Shortest path in maps (unweighted)
- Web crawlers
- Social network reach
- Network broadcasting
- Level order traversal in trees
- Minimum hops problems

Quick Summary

- BFS = Breadth-First Search
- Traverses graph **level by level**
- Uses Queue + visited[]
- Guarantees shortest path (unweighted)
- Time: **O(V + E)**
- Space: **O(V)**
- One of the most important graph algorithms

6.5 — Graph Traversal: Depth-First Search (DFS)

Go deep first, backtrack later.

1 What is DFS?

Depth-First Search (DFS) is a graph traversal technique where:

We explore **as far as possible along one path** before coming back (backtracking).

Instead of spreading out like BFS, DFS **dives deep** into the graph.

2️⃣ Why DFS is Needed

DFS is used when:

- You want to explore all possibilities
- You need **backtracking**
- You want to detect **cycles**
- You want to explore **connected components**
- You want to solve maze / puzzle / path existence problems

DFS is the backbone of:

- Backtracking
- Recursion-based algorithms
- Many graph problems (cycle detection, topological sort, etc.)

3️⃣ Core Idea (Intuition)

Think of DFS like exploring a maze:

- Choose one path
- Keep going forward
- If you hit a dead end → **go back**
- Try the next unexplored path

This "go deep, then backtrack" behavior is DFS.

4️⃣ DFS Traversal Order (Conceptual)

For a graph:

```
A — B — D
|   |
C   E
```

Possible DFS traversal starting from A:

A → B → D → (backtrack) → E → (backtrack) → C

Order depends on adjacency list order — DFS is **not unique**.

5️⃣ DFS Uses STACK (Explicit or Implicit)

DFS can be implemented in **two ways**:

1. Recursive DFS → uses call stack
2. Iterative DFS → uses explicit stack

Both follow the same logic.

6️⃣ DFS Algorithm (High Level Steps)

1. Start from a node
2. Mark it as visited
3. Process the node
4. For each unvisited neighbor:
    - Perform DFS on that neighbor

Repeat until all reachable nodes are visited.

7️⃣ DFS Using Adjacency List (Recursive)

Logic:

- Visit node
- Mark visited
- Recursively visit neighbors

Java Code — Recursive DFS

```
import java.util.*;

class GraphDFS {
    int V;
    ArrayList<ArrayList<Integer>> adj;

    GraphDFS(int V) {
        this.V = V;
        adj = new ArrayList<>();
        for (int i = 0; i < V; i++)
            adj.add(new ArrayList<>());
    }

    void addEdge(int u, int v) {
        adj.get(u).add(v);
        adj.get(v).add(u); // undirected graph
    }

    void dfs(int node, boolean[] visited) {
        visited[node] = true;
        System.out.print(node + " ");
        for (int neighbor : adj.get(node)) {
            if (!visited[neighbor]) {
                dfs(neighbor, visited);
            }
        }
    }
```

```
    public static void main(String[] args) {
        GraphDFS g = new GraphDFS(5);
        g.addEdge(0, 1);
        g.addEdge(0, 2);
        g.addEdge(1, 3);
        g.addEdge(1, 4);

        boolean[] visited = new boolean[5];
        g.dfs(0, visited);
    }
}
```

8 Dry Run (Recursive DFS)

Graph:

```
0
| \
1  2
| \
3  4
```

Start DFS from 0

Execution flow:

```
dfs(0)
 → dfs(1)
    → dfs(3)
    ← back
    → dfs(4)
    ← back
 ← back
 → dfs(2)
```

Output:

0 1 3 4 2

Notice the deep dive + backtracking.

9 DFS Using Stack (Iterative)

Logic:

- Use a stack explicitly
- Push start node
- Pop, process, push neighbors

Java Code — Iterative DFS

```
void dfsIterative(int start) {
    boolean[] visited = new boolean[V];
    Stack<Integer> stack = new Stack<>();

    stack.push(start);

    while (!stack.isEmpty()) {
        int node = stack.pop();

        if (!visited[node]) {
            visited[node] = true;
            System.out.print(node + " ");

            for (int neighbor : adj.get(node)) {
                if (!visited[neighbor]) {
                    stack.push(neighbor);
                }
            }
        }
    }
}
```

Stack mimics recursion manually.

10 Handling Visited Nodes (Very Important)

Without a visited[] array:

- DFS may go into **infinite loops**
- Especially in **cyclic graphs**

Rule:

Mark a node as visited **as soon as you visit it**

11 DFS Time & Space Complexity

Let:

- V = number of vertices
- E = number of edges

Time Complexity: O(V + E)

Space Complexity:

- Recursive DFS → O(V) (call stack)
- Iterative DFS → O(V) (explicit stack)
- Visited array → O(V)

1 2 DFS vs BFS (Intuition Comparison)

Aspect	BFS	DFS
Strategy	Level-wise	Depth-wise
Data structure	Queue	Stack / Recursion
Path guarantee	Shortest (unweighted)	No
Memory	Higher	Lower
Use case	Shortest paths	Exploration, backtracking

BFS = spread wide
DFS = go deep

1 3 Real-World Intuition & Applications

Maze Solving

- Try one path fully
- Backtrack if stuck

Cycle Detection

- Detect back edges

Backtracking Problems

- Sudoku
- N-Queens
- Word search

Connectivity Check

- Is graph fully connected?

Path Existence

- Is there a route from A to B?

1 4 DFS Mind Map

```
DFS
 ├── Go deep
 ├── Uses stack
 │    ├── Recursion
 │    └── Explicit Stack
 ├── Backtracking
 ├── Visited array
 ├── O(V + E)
```

```
└── Foundation for:
    ├── Cycle detection
    ├── Topological sort
    ├── Connected components
```

Quick Summary

- DFS explores **depth-first**
- Uses stack (implicit or explicit)
- Backtracking is key
- Traversal order depends on adjacency list
- Time complexity: **O(V + E)**
- Essential for maze, recursion, cycle detection

Mantra:

"BFS spreads knowledge. DFS seeks truth by going deep."

6.6 — BFS vs DFS (Conceptual Comparison)

Two ways to explore the same world — but with very different mindsets.

1️⃣ What Does "Traversal" Mean in Graphs?

Traversal means:

Visiting **every vertex** of a graph **systematically**,
without missing nodes or getting stuck in cycles.

Graphs are **non-linear** → no natural start/end like arrays or trees.
So we need **strategies**.

The two fundamental strategies are:

- BFS — Breadth First Search
- DFS — Depth First Search

2️⃣ Core Idea: BFS vs DFS in One Line

Traversal	Core Philosophy
BFS	Explore level by level
DFS	Explore depth by depth

- BFS is **wide-first**
- DFS is **deep-first**

3️⃣ Data Structure Used

Traversal	Data Structure
BFS	**Queue** (FIFO)
DFS	**Stack** (explicit) or **Recursion stack**

This directly connects Unit 4 (Stacks & Queues) with Graphs.

4️⃣ Traversal Order

BFS Traversal Order

Start → Visit all immediate neighbors

→ Then neighbors of neighbors

→ And so on…

Think:

"Who is closest first?"

DFS Traversal Order

Start → Go deep into one path

→ Keep going until stuck

→ Backtrack

→ Try another path

Think: "Go as far as possible before turning back."

5️⃣ Visual Intuition

BFS (Level-wise)

```
        A
      /   \
     B     C
    / \     \
   D   E     F
```

BFS Order: A → B → C → D → E → F

DFS (Depth-wise)

```
        A
      /   \
     B     C
    / \     \
   D   E     F
```

DFS Order: A → B → D → E → C → F

Same graph.
Completely different exploration behavior.

6️⃣ Memory Behavior (Key Difference)

Aspect	BFS	DFS
Memory usage	Higher (stores full levels)	Lower (stores path only)
Worst-case memory	O(V)	O(V)
Practical memory	More	Less

BFS can consume **more memory** for wide graphs.
DFS is more memory-efficient in sparse/deep graphs.

7️⃣ Handling Cycles (Both Need This!)

Graphs can have **cycles**.

Both BFS and DFS must track visited nodes, otherwise:

- Infinite loop
- Stack overflow
- Repeated processing

Conceptually:

visited[] array or set

8️⃣ Use Cases — When to Use BFS

Use **BFS** when:

✔️ You need shortest path in an unweighted graph

✔️ You need level-wise processing

✔️ You want minimum number of edges

✔️ You are solving:

- Social network distance
- Minimum hops
- Broadcasting
- Level-order exploration

BFS always finds the **shortest path first** (unweighted).

9️⃣ Use Cases — When to Use DFS

Use **DFS** when:

✔️ You need to explore all possibilities

✔️ You need backtracking

✔️ You want to check connectivity

✔️ You are solving:

- Maze problems
- Cycle detection
- Topological sorting
- Path existence
- Component discovery

DFS is excellent for decision trees and exploration problems.

🔟 Real-World Intuition

Scenario	Best Choice	Why
Find shortest route on map	BFS	Minimum steps
Maze solving	DFS	Deep exploration
Social network levels	BFS	Friends → friends of friends
Puzzle solving	DFS	Try → fail → backtrack
Web crawling	BFS	Level-based discovery
Dependency resolution	DFS	Deep resolution

1️⃣1️⃣ BFS vs DFS — Side-by-Side Summary Table

Feature	BFS	DFS
Traversal style	Level by level	Depth first
Data structure	Queue	Stack / Recursion
Memory usage	Higher	Lower
Shortest path	✔️ Yes (unweighted)	No
Backtracking	No	✔️ Yes
Best for	Distance, levels	Exploration, cycles
Implementation	Iterative	Recursive / Iterative
Risk	High memory	Stack overflow (deep graphs)

1️⃣2️⃣ Common Student Confusion

"Which one is faster?"
Both are **O(V + E)**

"Which one is better?"
Depends on the problem, not preference.

"Do companies expect both?"
YES. Understanding the **difference** matters more than memorizing code.

1️⃣3️⃣ Conceptual Mantra

BFS answers "How close?"
DFS answers "Is it possible?"

This single line clears 90% confusion.

Quick Summary

- BFS → Queue → Level-wise → Shortest path
- DFS → Stack/Recursion → Depth-wise → Exploration
- Both are essential
- Choice depends on **problem nature**
- This comparison is **interview gold**

6.7 — Basic Applications of Graphs

Where theory meets the real world.

Why Study Graph Applications?

Until now, we learned:

- What graphs are
- How they are represented
- How to traverse them (BFS, DFS)

Now comes the **WHY**.

Graphs are not just an academic structure —
they are the **backbone of modern systems**:

- Internet
- Google Maps
- Social media
- Compilers
- Networks
- Recommendation engines

1️⃣ Social Networks

Examples:

- Facebook
- Instagram
- LinkedIn
- Twitter (X)

Graph Model:

- Vertices (Nodes) → Users
- **Edges** → Friend / Follow relationship

Types:

- Undirected graph → Facebook (mutual friends)
- Directed graph → Twitter (followers)

Use of Traversal:

- BFS → mutual friends, degrees of separation
- DFS → community detection (conceptually)

"Friends of friends" is a graph traversal problem.

2️⃣ Road & Navigation Systems

Examples:

- Google Maps
- Apple Maps
- GPS systems

Graph Model:

- **Vertices** → Locations / junctions
- **Edges** → Roads
- **Weights** → Distance, time, traffic cost

Why Graph?

- Multiple paths between locations
- Need optimal routes

Traversal intuition:

- BFS → shortest path in unweighted maps
- DFS → exploring all routes (conceptual)

Every navigation app lives on a weighted graph.

3️⃣ Computer Networks

Examples:

- LAN, WAN
- Internet routing
- Data packet transmission

Graph Model:

- **Vertices** → Computers / routers
- **Edges** → Network connections

Why Graph?

- Data flows through multiple possible paths
- Failure of one node shouldn't collapse the system

Traversal intuition:

- BFS → broadcasting data
- DFS → deep inspection, fault detection

The Internet itself is a massive global graph.

4️⃣ Web Page Linking (World Wide Web)

Graph Model:

- **Vertices** → Web pages
- **Edges** → Hyperlinks

Type:

- Directed graph (link direction matters)

Applications:

- Page ranking
- Website crawling
- Link analysis

Traversal intuition:

- DFS → web crawling (depth exploration)
- BFS → level-wise crawling

Google Search was born from graph thinking.

5️⃣ Dependency Graphs

Examples:

- Software modules
- Package managers (npm, pip, maven)

- Build systems

Graph Model:

- **Vertices** → Tasks / modules
- **Edges** → Dependency relationship

Type:

- Directed graph

Why graph?

- A task can only run after dependencies finish

Traversal intuition:

- DFS → detect dependency chains
- Cycle detection → circular dependency errors

"Module A depends on B" = an edge in a graph.

6️⃣ Course Prerequisite Systems

Examples:

- University course planning
- Online learning platforms

Graph Model:

- Vertices → Courses
- **Edges** → Prerequisite relationship

Type:

- Directed graph
- Usually **acyclic**

Traversal intuition:

- DFS → prerequisite resolution
- BFS → semester-wise planning

You can't study Advanced DSA before Basics — graph rule!

7️⃣ Recommendation Systems (Intro Level)

Examples:

- Netflix
- Amazon
- Spotify
- YouTube

Graph Model:

- **Vertices** → Users + items
- **Edges** → Likes, views, purchases

Type:

- Bipartite graph (conceptual)

Traversal intuition:

- BFS → "Users like you also liked…"
- Graph distance → similarity score

Recommendations are graph relationships, not magic.

8️⃣ Network Routing (Conceptual)

Examples:

- Data packet routing
- Telecom networks

Graph Model:

- Vertices → Routers
- **Edges** → Communication links

Why graph?

- Multiple routes
- Fault tolerance
- Load balancing

Traversal intuition:

- BFS → shortest hop routing
- DFS → deep path exploration

Every packet chooses a path in a graph.

9️⃣ Key Insight: Why Graphs Are Powerful

Graphs handle:

- Relationships
- Connections
- Paths
- Dependencies
- Networks

They are perfect when:

- Data is not linear
- Multiple connections exist

- Hierarchy is insufficient (trees fail)

Whenever "everything connects to everything", think graph.

🔟 Conceptual Mapping (Big Picture)

Real World	Graph Concept
Friends	Nodes + edges
Roads	Weighted edges
Internet	Large graph
Dependencies	Directed edges
Recommendations	Bipartite graph
Routing	Shortest paths

Quick Summary

- Graphs model relationships and networks
- Used in:
  - Social media
  - Maps
  - Networks
  - Web
  - Dependencies
  - Recommendations
- BFS & DFS give **ways to explore**
- No advanced algorithms yet — just intuition
- Graphs solve problems trees and arrays cannot

Mantra:

"If data connects in many directions, graphs are the language of truth."

6.8 — Graph Traversal Mindset & Patterns

How to think when the structure is a graph.

1️⃣ Why Graph Traversal Needs a Special Mindset

In trees:

- There is **one root**
- No cycles
- No revisits

In graphs:

- There may be **cycles**
- Multiple paths to the same node
- No guaranteed starting point
- Graph can be **disconnected**

Key realization:

Traversal in graphs is not just about visiting —
it's about **remembering where you've already been**.

2️⃣ The MOST IMPORTANT Idea: Visited Concept

Why do we need visited[]?

Because graphs can have:

- Cycles
- Back edges
- Multiple incoming edges

Without visited tracking:

- Infinite loops
- Repeated work
- Stack overflow
- Time limit exceeded

Visited Structure

- Usually an array or set
- Size = number of vertices

visited[v] = true → already explored

visited[v] = false → not yet explored

Golden Rule:

Mark a node as visited **the moment you enter it**,
not after finishing it.

3️⃣ Traversal Pattern: Core Template (Mental Model)

Every graph traversal (BFS / DFS) follows this **universal pattern**:

```
for each vertex v:
   if v is not visited:
      start traversal from v
```

This single loop handles:

- Disconnected graphs
- Multiple components
- Isolated nodes

4️⃣ Full Graph Traversal (Disconnected Graphs)

Problem:

What if the graph is NOT connected?

Example:

Component 1: 0 — 1 — 2

Component 2: 3 — 4

Component 3: 5

If you start traversal from 0, you will **miss**:

- Nodes 3, 4, 5

Correct Approach

```
for (int i = 0; i < V; i++) {
   if (!visited[i]) {
      BFS(i) or DFS(i)
   }
}
```

This ensures:

- Every node is visited
- All components are covered

5️⃣ Connected Components (Intro Pattern)

A connected component is:

A group of vertices where each vertex is reachable from every other vertex in that group.

Traversal Insight:

- Each time you start BFS/DFS from an unvisited node
- You discover one connected component

Example:

Graph has 3 components → BFS/DFS starts 3 times

This idea is used later in:

- Counting islands
- Network clustering

- Friend groups
- Provinces problems

6️⃣ Avoiding Infinite Loops (CRITICAL)

- Common beginner mistake
- visit node
- explore neighbors
- come back
- explore again
- repeat forever
- Correct thinking
    - Visit node
    - Mark as visited immediately
    - Never process it again

Rule to remember:

In graphs, a node is visited **once**,
but edges may be seen multiple times.

7️⃣ BFS vs DFS — Mindset Difference

BFS Thinks Like:

- "Who is closest?"
- "Level by level"
- "Minimum steps"
- Uses **Queue**

DFS Thinks Like:

- "How deep can I go?"
- "Explore fully, then backtrack"
- Uses Recursion / Stack

Before coding, ask:

Do I want **depth** or **distance**?

8️⃣ Visualization Tips (Very Important)

To understand traversal:

- Always draw the graph
- Label nodes clearly

- Use colors:
  - White → unvisited
  - Grey → visiting
  - Black → visited
- Track queue/stack state manually

Visualization turns fear into clarity.

9️⃣ Common Beginner Mistakes (Must Avoid)

- Forgetting visited array
- Marking visited too late
- Assuming graph is connected
- Confusing BFS and DFS use cases
- Mixing tree logic with graph logic
- Ignoring self-loops or parallel edges

Every graph bug comes from **wrong assumptions**, not syntax.

🔟 Graph Traversal Pattern Memory Map

```
Graph Traversal Thinking:
│
├── Is the graph connected?
│      ├── Yes → Start from one node
│      └── No  → Loop over all nodes
│
├── Use visited[] to avoid cycles
│
├── Choose traversal:
│      ├── BFS → Queue → shortest path
│      └── DFS → Stack/Recursion → depth, backtracking
│
└── Process node once
```

1️⃣1️⃣ How to Approach ANY Graph Problem

Before writing code, ask yourself:

1. Is the graph directed or undirected?
2. Is it weighted or unweighted?
3. Can there be cycles?
4. Is the graph connected?

5. What do I need:
    - Shortest path?
    - Reachability?
    - Exploration?
6. BFS or DFS?

This questioning is what interviewers really test.

Quick Summary — 6.8

- Graph traversal ≠ Tree traversal
- `visited[]` is mandatory
- Always handle disconnected graphs
- BFS → level-wise, shortest path
- DFS → depth, backtracking
- One traversal per connected component
- Visualization is your superpower

Mantra: "In graphs, remembering where you've been is more important than where you're going."

6.9 — Summary & Reflection (Graphs – Basics)

From connections to traversal thinking.

1️⃣ Graphs — The Big Picture

A graph is a non-linear data structure used to represent relationships between entities.

- Entities → Vertices (Nodes)
- Relationships → **Edges**

Graphs help us model real-world connections, not just hierarchy.

Trees are a **special case** of graphs — graphs are the **general form**.

2️⃣ Core Graph Terminology

Term	Meaning
Vertex (Node)	An entity
Edge	Connection between two vertices
Degree	Number of edges connected to a vertex
Path	Sequence of vertices connected by edges
Cycle	Path that starts and ends at the same vertex
Connected Graph	Path exists between every pair of vertices

Term	Meaning
Disconnected Graph	At least one vertex is isolated
Directed Graph	Edges have direction
Undirected Graph	Edges have no direction
Weighted Graph	Edges have weights
Unweighted Graph	All edges treated equally

3️⃣ Why Graphs Are Needed

Arrays & Linked Lists:

- Store **linear** data

Trees:

- Store hierarchical data

Graphs:

- Store networked / many-to-many relationships

When relationships are **complex and interconnected**, graphs are the natural choice.

4️⃣ Types of Graphs — Mental Map

- Directed vs Undirected
- Weighted vs Unweighted
- Cyclic vs Acyclic
- Connected vs Disconnected
- Complete Graph
- Bipartite Graph
- Tree → special acyclic connected graph

Knowing *types* helps decide *algorithms* later.

5️⃣ Graph Representations — Recap

Adjacency Matrix

- 2D matrix
- Fast edge lookup
- High memory usage

Adjacency List

- List of neighbors for each vertex
- Memory efficient
- Preferred for sparse graphs

Matrix vs List — Cheat Sheet

Feature	Adjacency Matrix	Adjacency List
Space	$O(V^2)$	$O(V + E)$
Edge check	$O(1)$	$O(deg(V))$
Sparse graph	Inefficient	Best
Dense graph	Suitable	⚠️ Less ideal
Common usage	Small graphs	Real-world graphs

6️⃣ Graph Traversal — What It Really Means

Traversal = Systematically visiting all vertices of a graph while avoiding infinite loops

Two fundamental traversal strategies:

- **BFS** → Level-wise exploration
- **DFS** → Depth-wise exploration

Traversal is the **foundation** of all graph algorithms.

7️⃣ BFS — Essence Recap

- Uses **Queue**
- Explores neighbors first
- Finds shortest path in unweighted graphs
- Level-by-level processing

Used in:

- Shortest path (unweighted)
- Network broadcasting
- Level-order exploration

8️⃣ DFS — Essence Recap

- Uses Recursion or Stack
- Goes as deep as possible first
- Excellent for exploration & backtracking

Used in:

- Maze solving
- Cycle detection
- Connectivity checking
- Backtracking problems

9️⃣ BFS vs DFS — One-Line Intuition

- BFS explores width first.
- DFS explores depth first.

Neither is better universally — **choice depends on the problem**.

🔟 Traversal Mindset (Most Important Learning)

✔ Always maintain a **visited array / set**
✔ Graphs may be **disconnected**
✔ Traversal may need to restart from multiple vertices
✔ Infinite loops happen if visited is ignored
✔ Think in terms of **movement, not values**

Graph problems are about **navigation**, not storage.

1️⃣1️⃣ Common Beginner Pitfalls (Awareness)

- Forgetting visited array
- Assuming graph is connected
- Confusing tree traversal with graph traversal
- Ignoring direction of edges
- Mixing BFS & DFS logic

Understanding these mistakes early builds **strong intuition**.

1️⃣2️⃣ Where Graphs Appear in Real Systems

- Social networks (friends, followers)
- Road maps & GPS navigation
- Computer networks & routing
- Web page linking
- Dependency systems
- Recommendation engines
- Scheduling systems

Graphs quietly power **modern technology**.

1️⃣3️⃣ What We Have NOT Done (Intentionally)

We **did not** cover:

- Shortest path algorithms
- Minimum Spanning Trees
- Topological sorting

- Cycle detection algorithms
- Graph coloring

These belong to advanced graph theory — coming next.

1️⃣4️⃣ Bridge to the Next Level

From here, graphs evolve into **problem-solving engines**:

- Shortest Paths → Dijkstra, BFS-based paths
- Network optimization → MST
- Dependency resolution → Topological Sort

This foundation ensures students don't **memorize algorithms** —
they **understand why they exist**.

1️⃣5️⃣ Final Reflection

Trees taught us **hierarchy**
Graphs teach us **relationships**

Graphs are not hard —
they are just **movement with discipline**.